Chapter 10 Thinking in Objects

Objectives

- To apply class abstraction to develop software (§10.2).
- To explore the differences between the procedural paradigm and object-oriented paradigm (§10.3).
- To discover the relationships between classes (§10.4).
- To design programs using the object-oriented paradigm (§10.5–10.6).
- To create objects for primitive values using the wrapper classes (**Byte**, **Short**, **Integer**, **Long**, **Float**, **Double**, **Character**, and **Boolean**) (§10.7).
- To simplify programming using automatic conversion between primitive types and wrapper class types (§10.8).

Immutable Objects and Classes

- If the contents of an object cannot be changed once the object is created, the object is called an *immutable object* and its class is called an *immutable class*.

WHEN??

A class with all private data fields
NO set method

A class with all private data fields and without mutators (Set) is not necessarily immutable.

IS it right???

Immutable Objects and Classes

- If you delete the set method in the CircleWithPrivateDataFields class, the class would be immutable because radius is private and cannot be changed without a set method.

```
public class CircleX {

private double radius = 1;

private static int numberOfObjects = 0;

public CircleX() {
        numberOfObjects++;
}


CircleX(double newRadius) {
radius = newRadius;
numberOfObjects++;
}

public double getRadius() {
    return radius;
}
```

```
public void setRadius(double newRadius) {
radius = (newRadius >= 0) ? newRadius : 0
}

public static int getNumberOfObjects() {
return numberOfObjects;
}

public double getArea() {
return radius * radius * Math.PI;
}

public static void main(String[] args) {

CircleX t = new CircleX(10);


}
 }
```

Example

```
public class Student {
  private int id ;
  private BirthDate birthDate;

public Student(int ssn, int year, int month, int day){
    id = ssn;
    birthDate = new BirthDate(year, month, day);
  }

  public int getId() {
    return id;
  }

  public BirthDate    getBirthDate() {
    return birthDate;
  }
}
```

```
public class BirthDate {
  private int year;
  private int month;
  private int day;

public BirthDate(int newYear,
      int newMonth, int newDay){
    year = newYear;
    month = newMonth;
    day = newDay;
  }

public void setYear(int newYear){
    year = newYear;
  }
}
```

```
public class Test {
  public static void main(String[] args) {
    Student student = new Student(1212, 1970, 5, 3);
    BirthDate date = student.getBirthDate();
    date.setYear(2010); // Now the student birth year is changed!
  }
}
```

When is a Class Immutable?

For a class to be immutable, it **must** :

- All data fields must be private.
- Provide no mutator (set) methods for data fields.
- Provide no accessor (get)methods that return a reference to a mutable data field object.

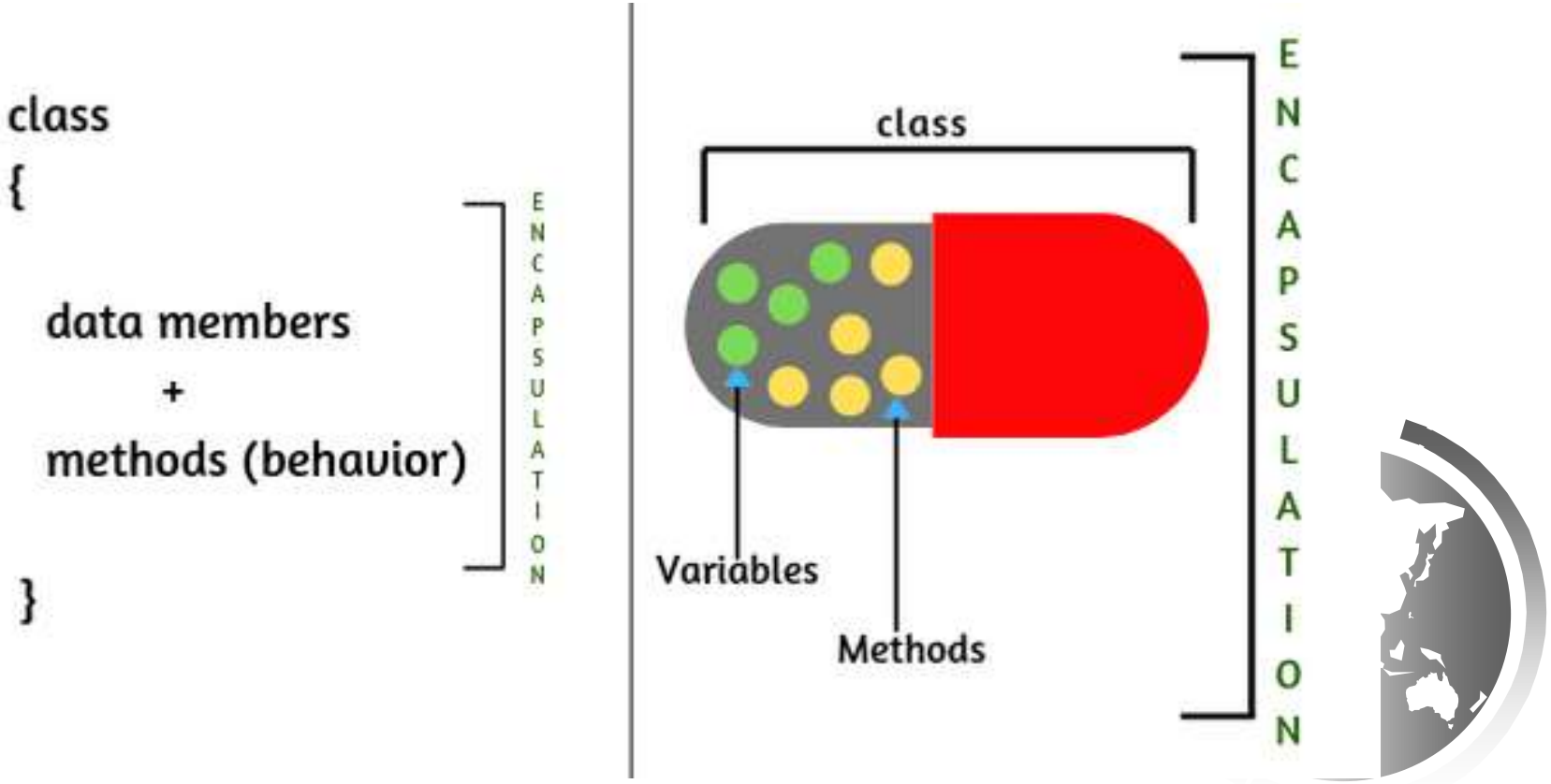
ENCAPSULATION:

Data is Hiden / Details are Hiden

Main objects contain both
the attribute and behaviors of
the encapsulated objects but cannot access
their algorithms
or internal attributes.

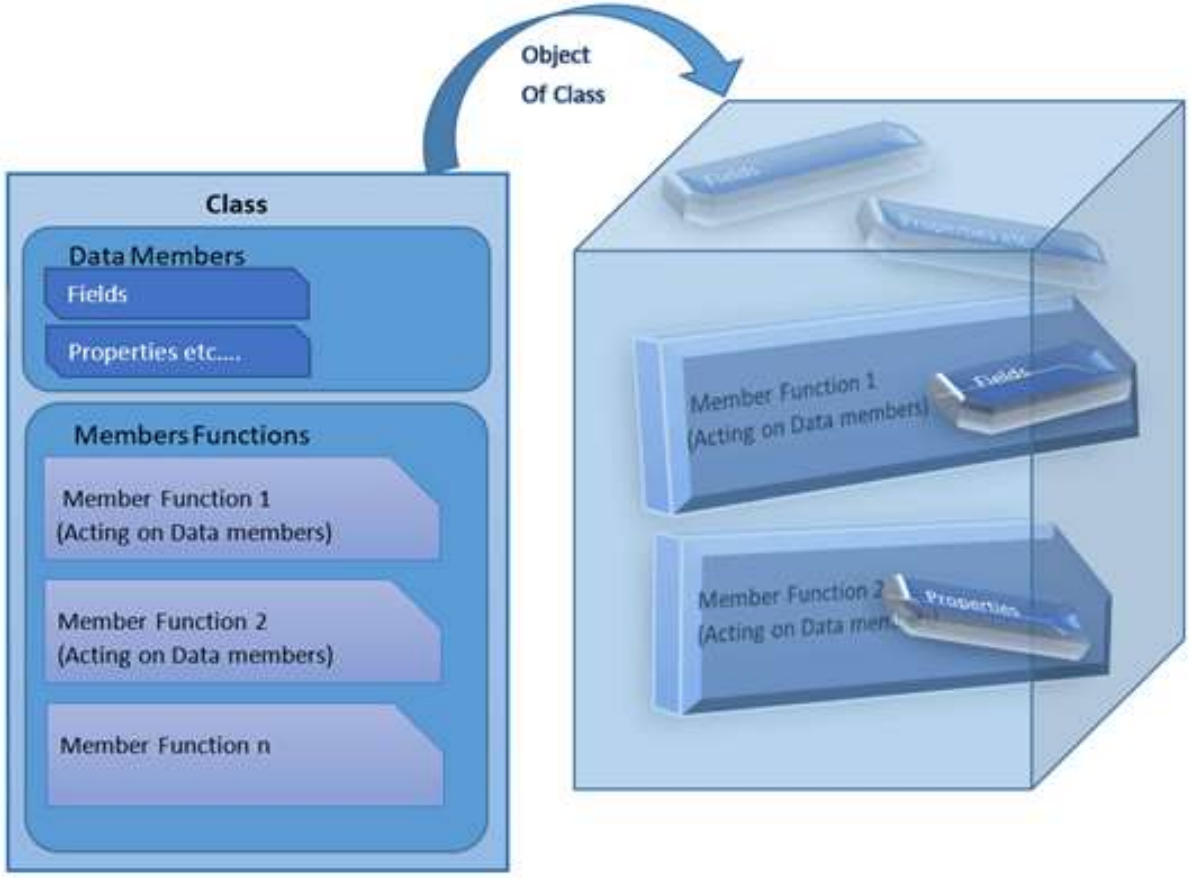
Class Encapsulation

Class Encapsulation means hiding the details of the implementation from the user.



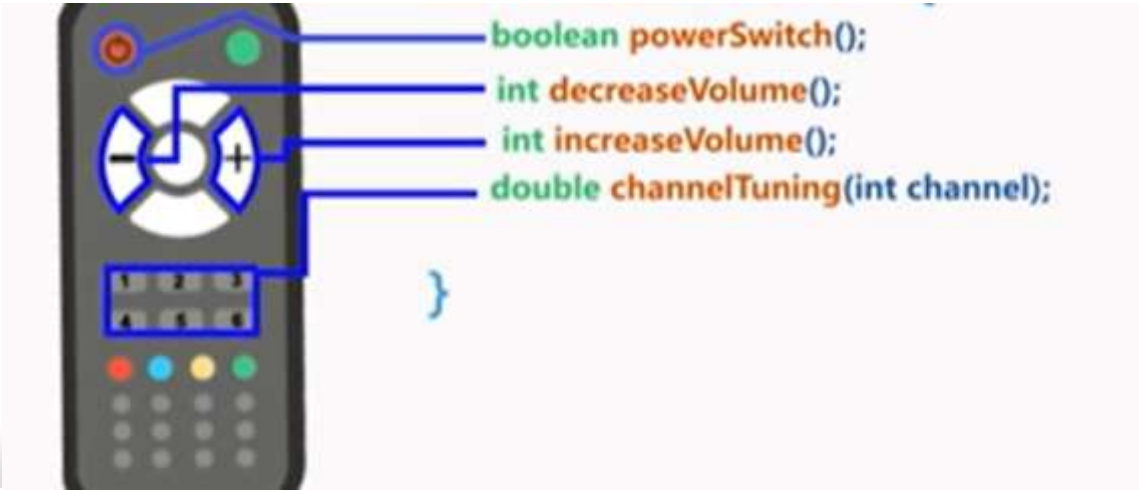
Class Abstraction

Class Abstraction means separating the class implementation from the use of the class.



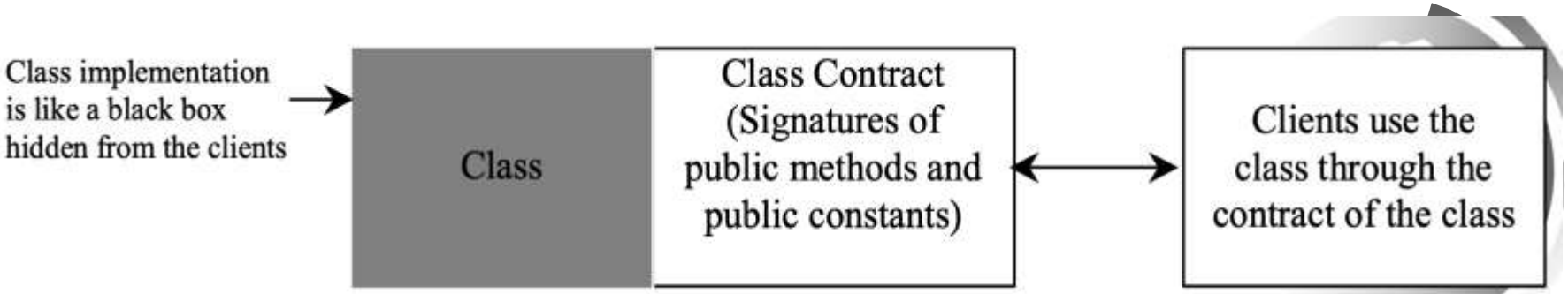
Example


Outside Class
CLASS
Mobile
Public
Instance
Variables
Private
Instance Variables
Private Methods
Public
Methods
Receive()
Dial()


boolean powerSwitch();
int decreaseVolume();
int increaseVolume();
double channelTuning(int channel);
}

Class Abstraction and Encapsulation

- The creator of the class provides a description of the class and the user of the class does not need to know how the class is implemented.
- The Signatures of public methods and fields that are accessible from outside the class, together with the description of how these members are expected to behave, serves as the Class's Contract.
- The details of the implementation is encapsulated and hidden from the user.



Designing the Loan Class

Loan	
-annualInterestRate: double	The annual interest rate of the loan (default: 2.5).
-numberOfYears: int	The number of years for the loan (default: 1)
-loanAmount: double	The loan amount (default: 1000).
-loanDate: Date	The date this loan was created.
+Loan()	Constructs a default Loan object.
+Loan(annualInterestRate: double, numberOfYears: int, loanAmount: double)	Constructs a loan with specified interest rate, years, and loan amount.
+getAnnualInterestRate(): double	Returns the annual interest rate of this loan.
+getNumberOfYears(): int	Returns the number of the years of this loan.
+getLoanAmount(): double	Returns the amount of this loan.
+getLoanDate(): Date	Returns the date of the creation of this loan.
+setAnnualInterestRate(annualInterestRate: double): void	Sets a new annual interest rate to this loan.
+setNumberOfYears(numberOfYears: int): void	Sets a new number of years to this loan.
+setLoanAmount(loanAmount: double): void	Sets a new amount to this loan.
+getMonthlyPayment(): double	Returns the monthly payment of this loan.
+getTotalPayment(): double	Returns the total payment of this loan.

The object-oriented programming paradigm focuses on objects, and actions are defined along with the data in objects.

serves as the contract for the **Loan** class

The tasks

attributes

2. Declare the constructors
3. Define the getter and setter methods
4. Define the service methods
   - Method to calculate the monthly payment
   - Method to calculate the Total payment
5. Define the application class : Input the data (yearly interest, number of years for the loan, loan amount)
6. Define the application class:
   - Create A Loan Object
   - Display loan Date , Monthly payment, Total Payment.

The attributes

```
public class Loan {
  private double annualInterestRate;
  private int numberOfYears;
  private double loanAmount;
  private java.util.Date loanDate;
```

The constructors

```
/** Default constructor */
public Loan() {
  this(2.5, 1, 1000);
}

/** Construct a loan with specified annual interest rate,
    number of years, and loan amount
  */
public Loan(double annualInterestRate, int numberOfYears,
    double loanAmount) {
  this.annualInterestRate = annualInterestRate;
  this.numberOfYears = numberOfYears;
  this.loanAmount = loanAmount;
  loanDate = new java.util.Date();
}
```

The getters and the setters

```
/** Return annualInterestRate */
public double getAnnualInterestRate() {
  return annualInterestRate;
}

/** Set a new annualInterestRate */
public void setAnnualInterestRate(double annualInterestRate) {
  this.annualInterestRate = annualInterestRate;
}

/** Return numberOfYears */
public int getNumberOfYears() {
  return numberOfYears;
}

/** Set a new numberOfYears */
public void setNumberOfYears(int numberOfYears) {
  this.numberOfYears = numberOfYears;
}

/** Return loanAmount */
public double getLoanAmount() {
  return loanAmount;
}

/** Set a newloanAmount */
public void setLoanAmount(double loanAmount) {
  this.loanAmount = loanAmount;
}
```

```
/** Return loan date */
public java.util.Date getLoanDate() {
  return loanDate;
}
```

Service Methods

- Method to calculate the monthly payment
- Method to calculate the Total payment

```
/** Find monthly payment */
public double getMonthlyPayment() {
  double monthlyInterestRate = annualInterestRate / 1200;
  double monthlyPayment = loanAmount * monthlyInterestRate / (1 -
    (1 / Math.pow(1 + monthlyInterestRate, numberOfYears * 12)));
  return monthlyPayment;
}

/** Find total payment */
public double getTotalPayment() {
  double totalPayment = getMonthlyPayment() * numberOfYears * 12;
  return totalPayment;
}
```

$$monthlyPayment = \frac{loanAmount \times monthlyInterestRate}{1 - (1 + monthlyInterestRate)^{-numberOfYears \times 12}}$$

The Application class

- Input the data (yearly interest, number of years for the loan, loan amount)

```
import java.util.Scanner;

public class TestLoanClass {
  /** Main method */
  public static void main(String[] args) {
    // Create a Scanner
    Scanner input = new Scanner(System.in);

    // Enter yearly interest rate
    System.out.print(
      "Enter yearly interest rate, for example, 8.25: ");
    double annualInterestRate = input.nextDouble();

    // Enter number of years
    System.out.print("Enter number of years as an integer: ");
    int numberOfYears = input.nextInt();

    // Enter loan amount
    System.out.print("Enter loan amount, for example, 120000.95: ");
    double loanAmount =  input.nextDouble();
```

The Application class

- Create A Loan Object
- Display loan Date , Monthly payment, Total Payment.

```
// Create Loan object
Loan loan =
  new Loan(annualInterestRate, numberOfYears, loanAmount);

// Display loan date, monthly payment, and total payment
System.out.printf("The loan was created on %s\n" +
  "The monthly payment is %.2f\nThe total payment is %.2f\n",
  loan.getLoanDate().toString(), loan.getMonthlyPayment(),
  loan.getTotalPayment());
}
```

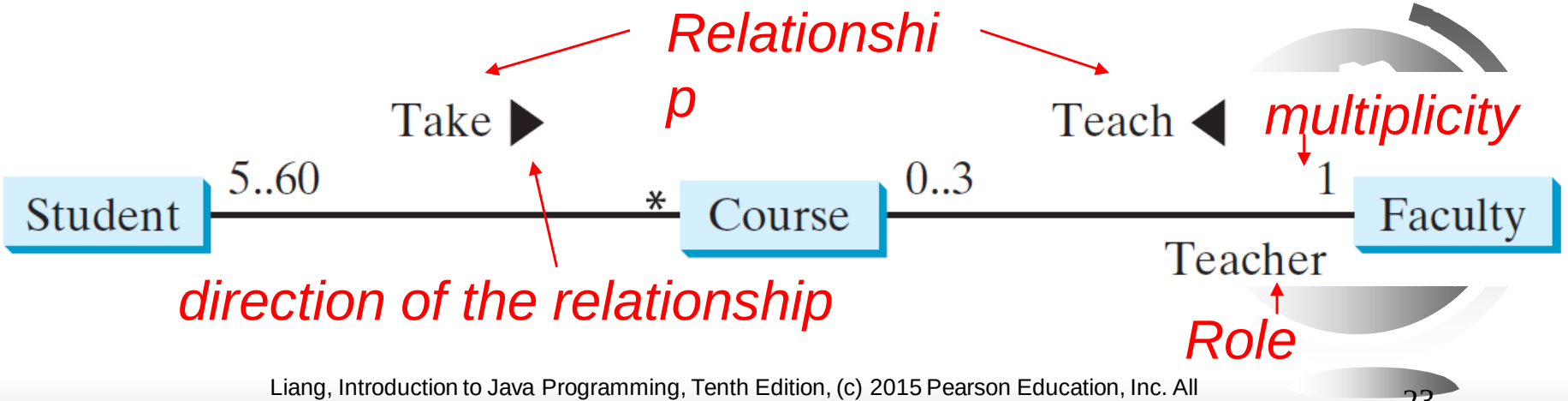
Abstraction	Encapsulation
1. Abstraction solves the problem in the design level.	1. Encapsulation solves the problem in the implementation level.
2. Abstraction is used for hiding the unwanted data and giving relevant data.	2. Encapsulation means hiding the code and data into a single unit to protect the data from outside world.
3. Abstraction lets you focus on what the object does instead of how it does it	3. Encapsulation means hiding the internal details or mechanics of how an object does something.
4. **Abstraction**- Outer layout, used in terms of design. For Example:- Outer Look of a Mobile Phone, like it has a display screen and keypad buttons to dial a number.	4. **Encapsulation**- Inner layout, used in terms of implementation. For Example:- Inner Implementation detail of a Mobile Phone, how keypad button and Display Screen are connect with each other using circuits.

Class Relationships

- Classes in a software system can have various types of relationships to each other.
- Three of the most common relationships:
  - Dependency: A *uses* B
  - Aggregation: A *has-a* B
  - Inheritance: A *is-a* B

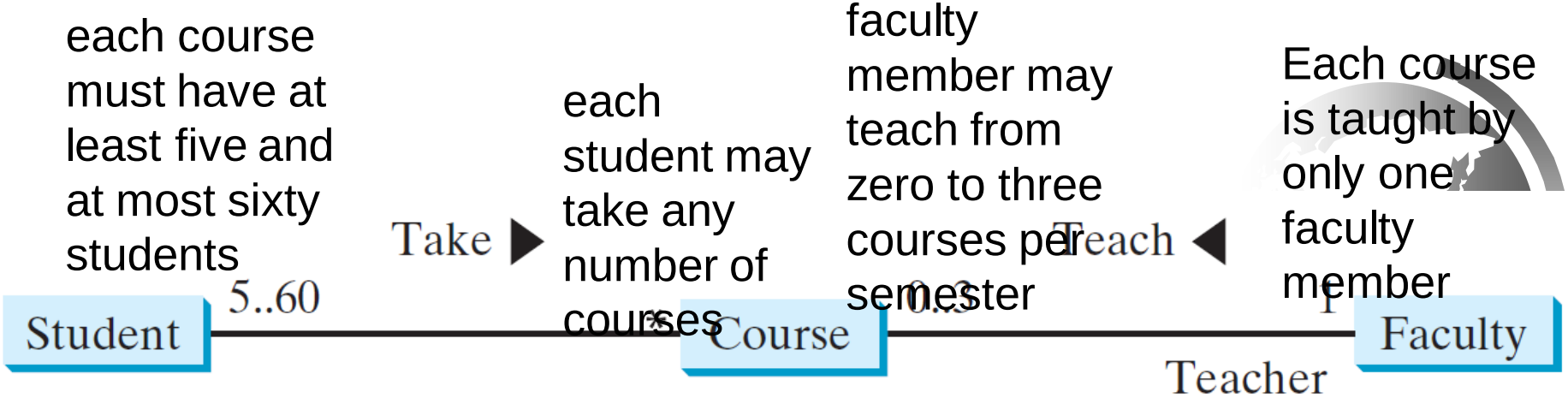
Association

- *Association* is a general binary relationship that describes an activity between two classes.
- For example:
  - A student taking a course is an association between the **Student** class and the **Course** class.
  - A faculty member teaching a course is an association between the **Faculty** class and the **Course** class.



Multiplicity

- The character * means an unlimited number of objects.
- The interval **m..n** indicates that the number of objects is between **m** and **n**, inclusively.



Implementing the Association Relations

- We can implement associations using data fields and methods.
- The relation "a student takes a course" is implemented using the **addCourse** method in the **Student** class and the **addStudent** method in the **Course** class.
- The relation "a faculty teaches a course" is implemented using the **addCourse** method in the **Faculty** class and the **setFaculty** method in the **Course** class.

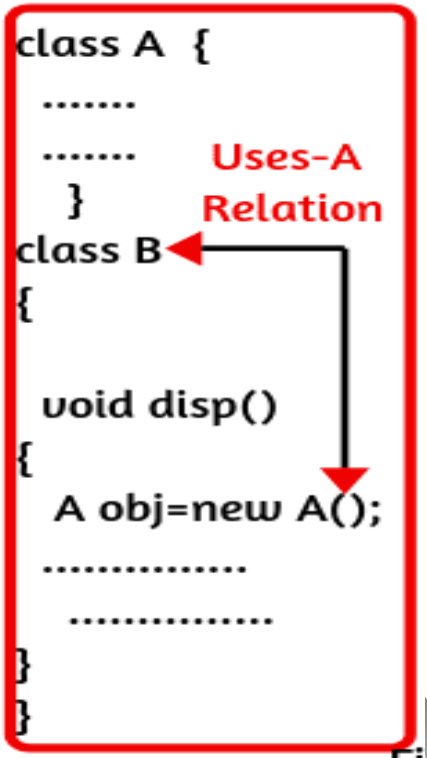
```
public class Student {
  private Course[]
    courseList;

  public void addCourse(
    Course s) { ... }
}
```

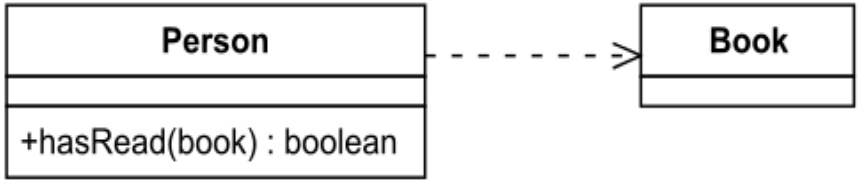
```
public class Course {
  private Student[]
    classList;
  private Faculty faculty;

  public void addStudent(
    Student s) { ... }

  public void setFaculty(
    Faculty faculty) { ... }
}
```

```
public class Faculty {
  private Course[]
    courseList;

  public void addCourse(
    Course c) { ... }
}
```

Dependency

- A *dependency* exists when one class relies on another in some way, usually by invoking the methods of the other.
- We've seen dependencies in many previous examples.
- Symbolized by an arrow pointing from the dependent class to the independent class.





Dependency

```
class Printer {
   public void print(String s) {
      System.out.println(s);
   }
}

class Student {
   public void show() {
Printer p = new Printer();
      p.print("Hello");
   }
}
```

```
public class TestLoanClass {
public static void main(String[] args) {
// Create Loan object
   Loan loan =new Loan(annualInterestRate,
numberOfYears, loanAmount);

     System.out.printf("The loan was created on %s\n" +
     "The monthly payment is %.2f\nThe total payment is
%.2f\n",
     loan.getLoanDate().toString(),
loan.getMonthlyPayment(),
     loan.getTotalPayment());
  }
}
```

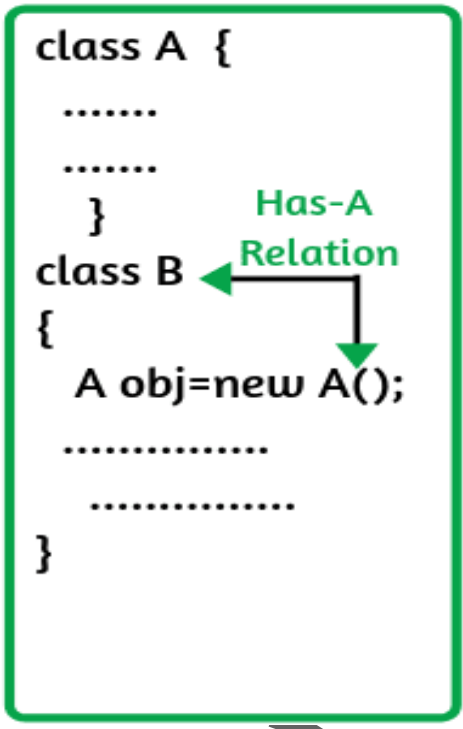
Aggregation

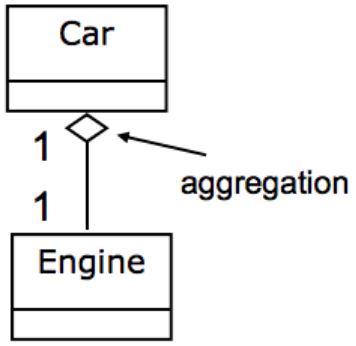
- *Aggregation* is a special form of association that represents an ownership relationship between two objects.
- Aggregation is a *has-a* or *part-of* relationship.

A car *has an engine*

A car has a weel

- An aggregating object **contains references to other objects** as **instance data.**
- The subject object is called the aggregated object.
- The owner object is called an aggregating object.
- Symbolized by a clear white diamond.





Aggregation

```
public class Student {
  private int id;
  private BirthDate birthDate;

  public Student(int ssn,
      int year, int month, int day) {
    id = ssn;
    birthDate = new BirthDate(year, month, day);
  }

  public int getId() {
    return id;
  }

  public BirthDate getBirthDate() {
    return birthDate;
  }
}
```

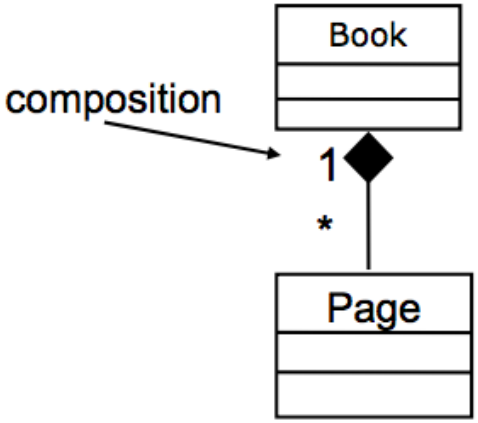
```
class Engine {
. . . . . .
}

class Car {
  Engine engine=new Engine();
 // has-a relationship
}
```

Composition

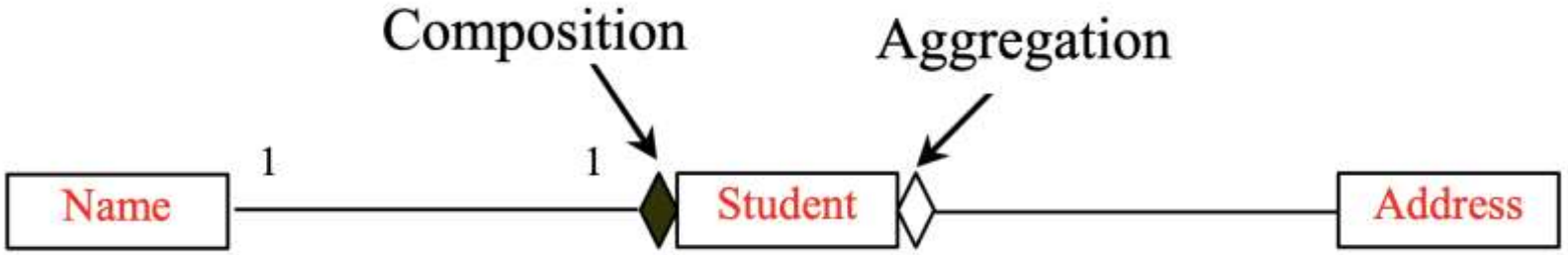
- If an object is **exclusively owned by an aggregating** object, this relation is called composition.
- Composition is a stronger version of aggregation.
  - the parts live and die with the whole.
  - symbolized by a black diamond.

Examples:

- A student has a name
- A book has pages
- A building has concrete blocks



Aggregation and Composition



- "A student has a Name", If an object is exclusively owned by an aggregating object, **Composition.**
- But "A student has an Address", an address could be shared by more than one student, **Aggregation.**

Implementation of Aggregation and Composition Relations

An aggregation relationship is usually represented as a data field in the aggregating class.

The relation "a student has a name" and "a student has an address" are implemented in the data field **name** and **address** in the **Student** class.

```
public class Name {
  ...
}
```

Aggregated class

```
public class Student {
  private Name name;
  private Address address;

  ...
}
```

Aggregating class

```
public class Address {
  ...
}
```

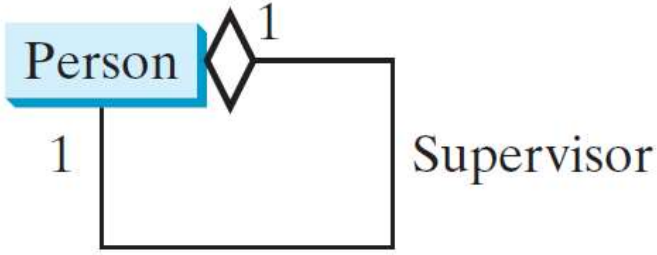
Aggregated class

The subject object is called the aggregated object

The owner object is called an aggregating object. OR is an object that is made up of other objects

Aggregation Between Same Class

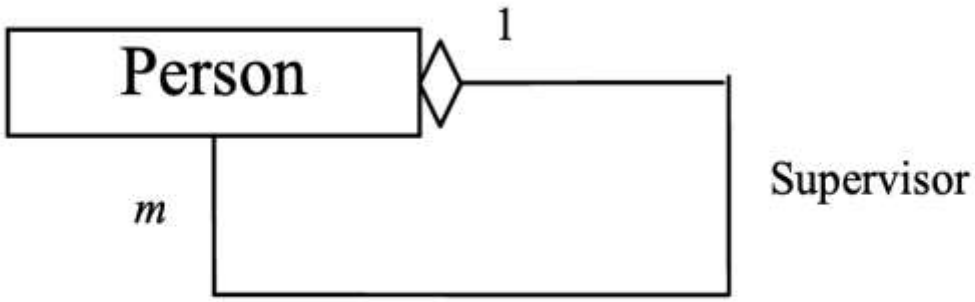
Aggregation may exist between objects of the same class. For example, a person may have a supervisor.




```
public class Person {
  // The type for the data is the class itself
  private Person supervisor;
  ...
}
```

Aggregation Between Same Class

What happens if a person has several supervisors?




```
public class Person {
  ...
  private Person[] supervisors;
}
```

Example: The Course Class

Course
-courseName: String -students: String[] -numberOfStudents: int
+Course(courseName: String) +getCourseName(): String +addStudent(student: String): void +dropStudent(student: String): void +getStudents(): String[] +getNumberOfStudents(): int

Course

TestCourse

Run

Example

```
public class TestCourse {
  public static void main(String[] args) {
    Course course1 = new Course("Data Structures");
    Course course2 = new Course("Database Systems");

    course1.addStudent("Peter Jones");
    course1.addStudent("Brian Smith");
    course1.addStudent("Anne Kennedy");

    course2.addStudent("Peter Jones");
    course2.addStudent("Steve Smith");

    System.out.println("Number of students in course1:
      + course1.getNumberOfStudents());
    String[] students = course1.getStudents();
    for (int i = 0; i < course1.getNumberOfStudents();
      System.out.print(students[i] + ", ");

    System.out.println();
    System.out.print("Number of students in course2: "
      + course2.getNumberOfStudents());
  }
}
```

```
public class Course {
  private String courseName;
  private String[] students = new String[100];
  private int numberOfStudents;

  public Course(String courseName) {
    this.courseName = courseName;
  }

  public void addStudent(String student) {
    students[numberOfStudents] = student;
    numberOfStudents++;
  }

  public String[] getStudents() {
    return students;
  }

  public int getNumberOfStudents() {
    return numberOfStudents;
  }

  public String getCourseName() {
    return courseName;
  }

  public void dropStudent(String student) {
    // Left as an exercise in Exercise 10.9
  }
}
```

- Should a Student be represented as a String type only?
- How can you modify the code?
- Write the implementation of the dropStudent method.

The dropStudent method

```
public void dropStudent(String student) {
   for(int i=0; i<numberOfStudents; i++) {
         if(students[i].equals(student)) {
         for (int j = i; j < numberOfStudents - 1; j++) {
            students[j] = students[j + 1];
         }
         students[numberOfStudents - 1] = null;
         numberOfStudents--;
         break;
      }
   }}
```

Test Your Knowledge

Draw the class relationships in the following UML diagram

Name

-firstName: String
-mi: char
-lastName: String

+Name()
+Name(firstName: String, mi: char, lastName: String)
+getFirstName(): String
+getMi(): char
+getLastName(): String
+setFirstName(firstName: String): void
+setMi(mi: char): void
+setLastName(lastName: String): void
+getFullName(): String

Person

-name: Name
-address: Address

+Person()
+Person(name: Name, address: Address)
+getName(): Name
+seName(name: Name): void
+getAddress(): Address
+setAddress(address: Address): void
+toString(): String

Address

-street: String
-city: String
-state: String
-zip: String

+Address()
+Address(street: String, city: String, state: String, zip: String)
+getStreet(): String
+getCity(): String
+getState(): String
+getZip(): String
+setStreet(street: String): void
+setCity(city: String): void
+setState(state: String): void
+setZip(zip: String): void
+getFullAddress(): String

Loan

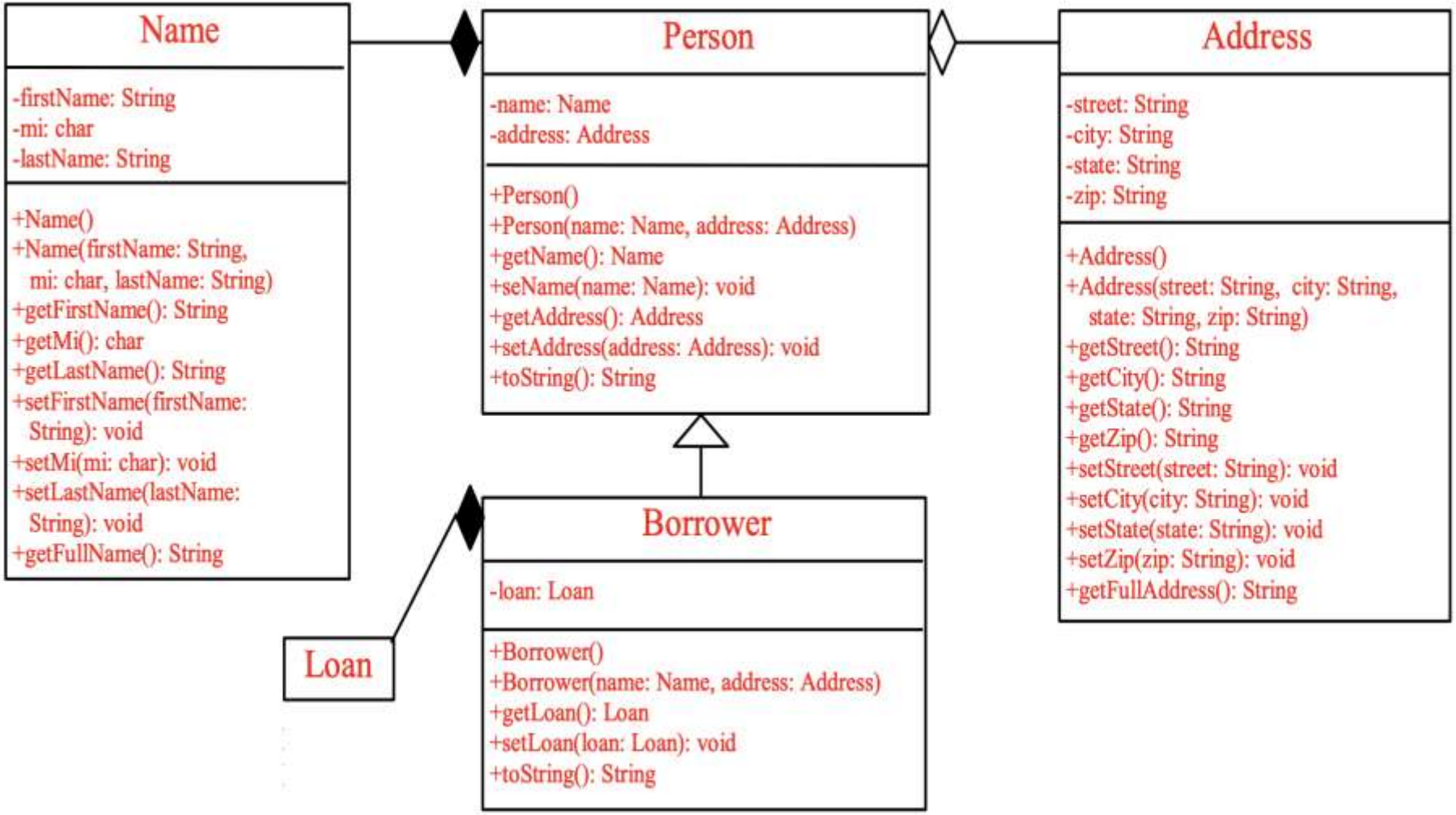
Borrower

-loan: Loan

+Borrower()
+Borrower(name: Name, address: Address)
+getLoan(): Loan
+setLoan(loan: Loan): void
+toString(): String

Test Your Knowledge

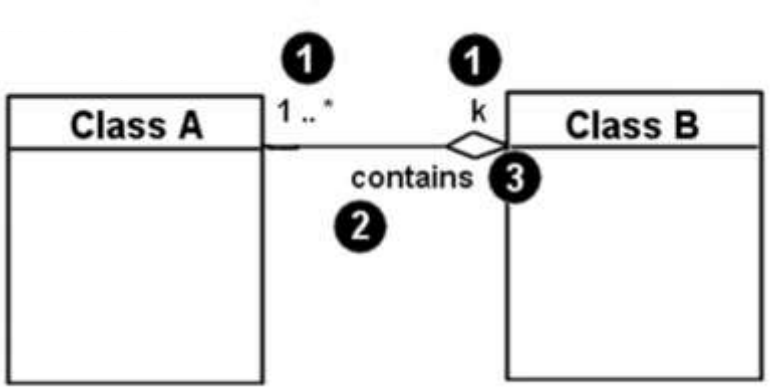
Draw the class relationships in the following UML diagram



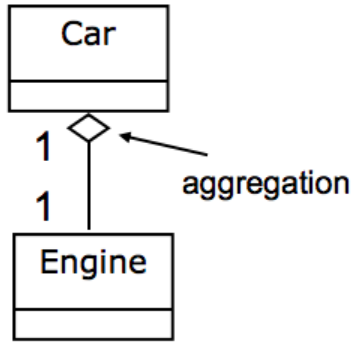
Revision of Class Relationships

Associational (usage) relationships

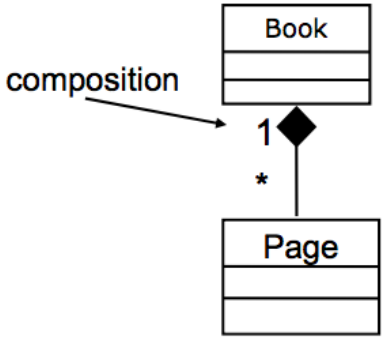
1. Multiplicity (how many are used)
   - * ⇒ 0, 1, or more
   - 1 ⇒ 1 exactly
   - 2..4 ⇒ between 2 and 4, inclusive
   - 3..* ⇒ 3 or more (also written as "3..")
2. name (what relationship the objects have)
3. navigability (direction)



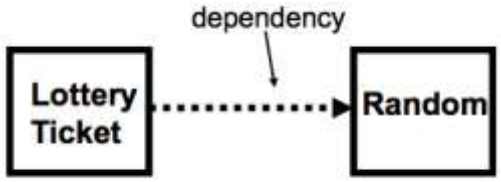
Revision of Classes Relationships



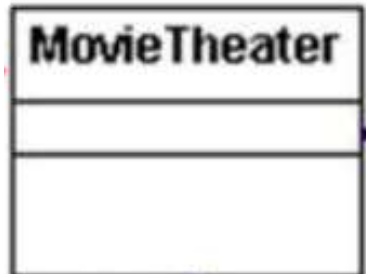
- Aggregation: "is part of"
  - symbolized by a clear white diamond
- Composition: "is entirely made of"
  - stronger version of aggregation
  - the parts live and die with the whole
  - symbolized by a black diamond

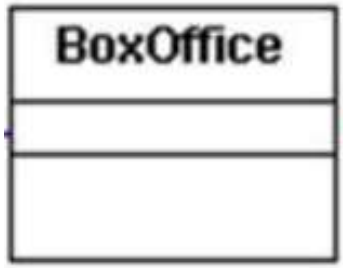


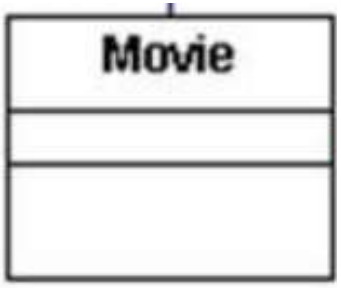
- Dependency: "uses temporarily"
  - symbolized by dotted line
  - often is an implementation detail, not an intrinsic part of that object's state



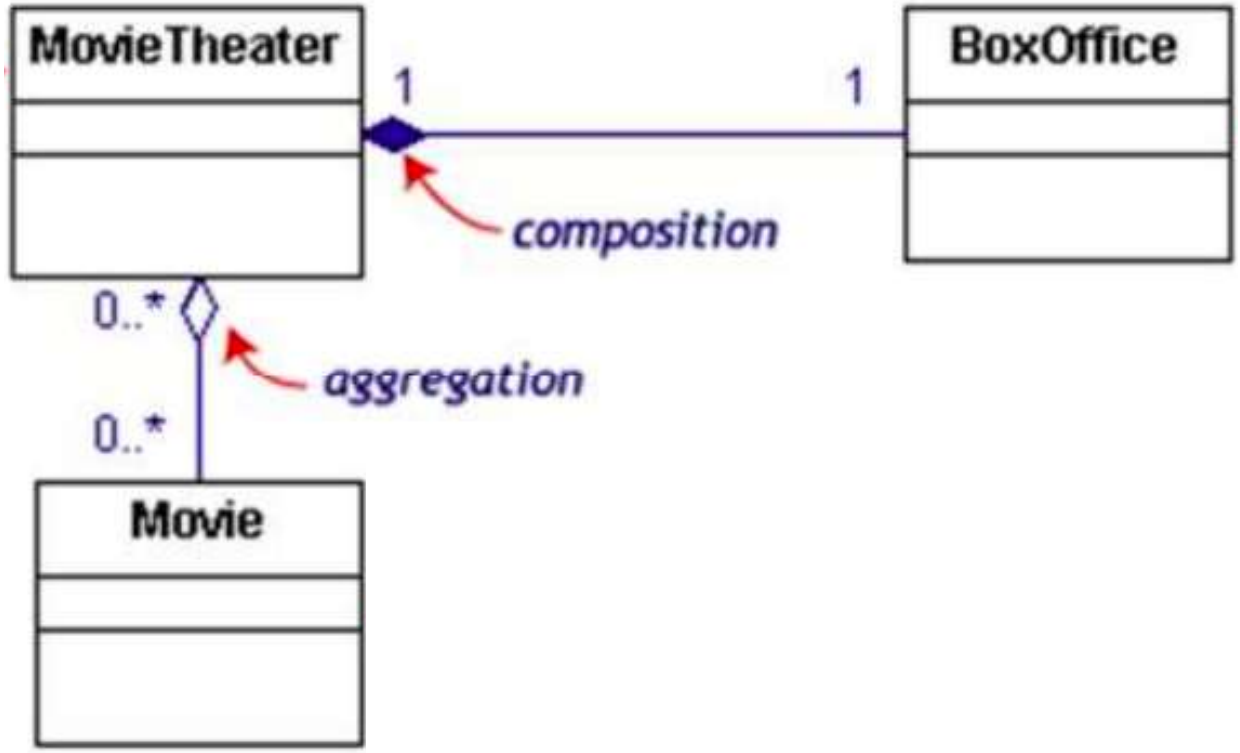
Draw the class relationships in the following UML diagram





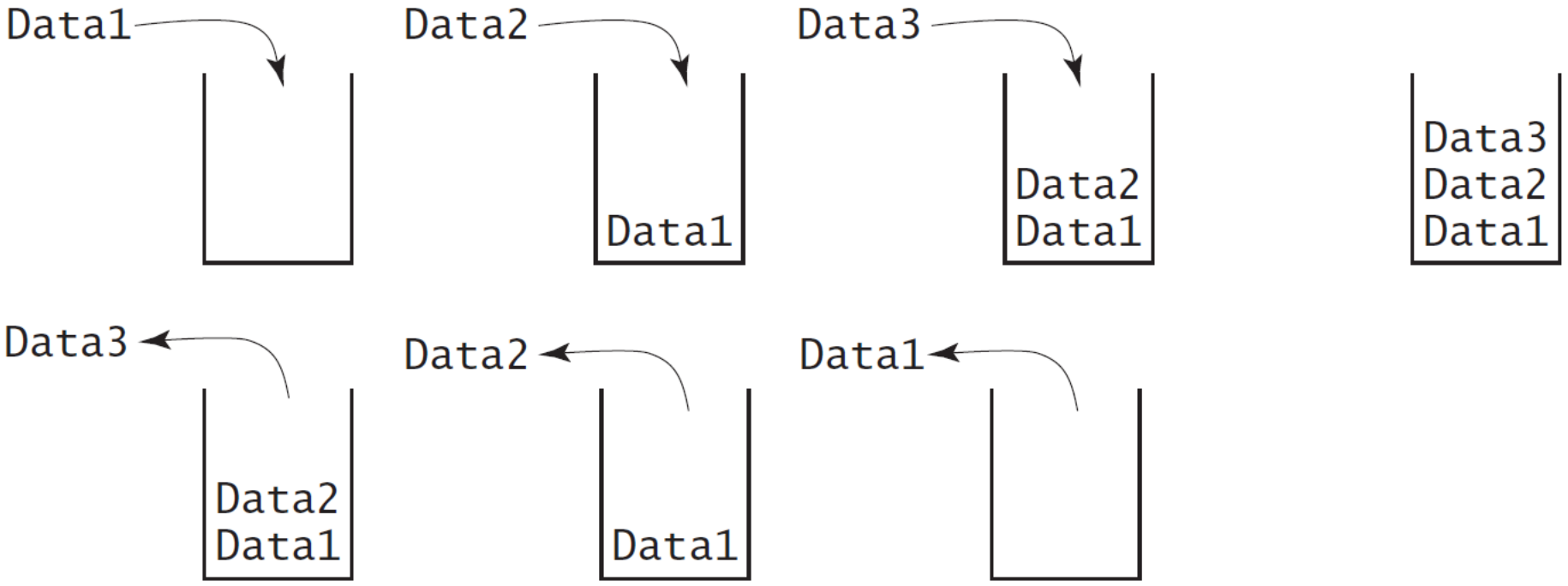


Draw the class relationships in the following UML diagram

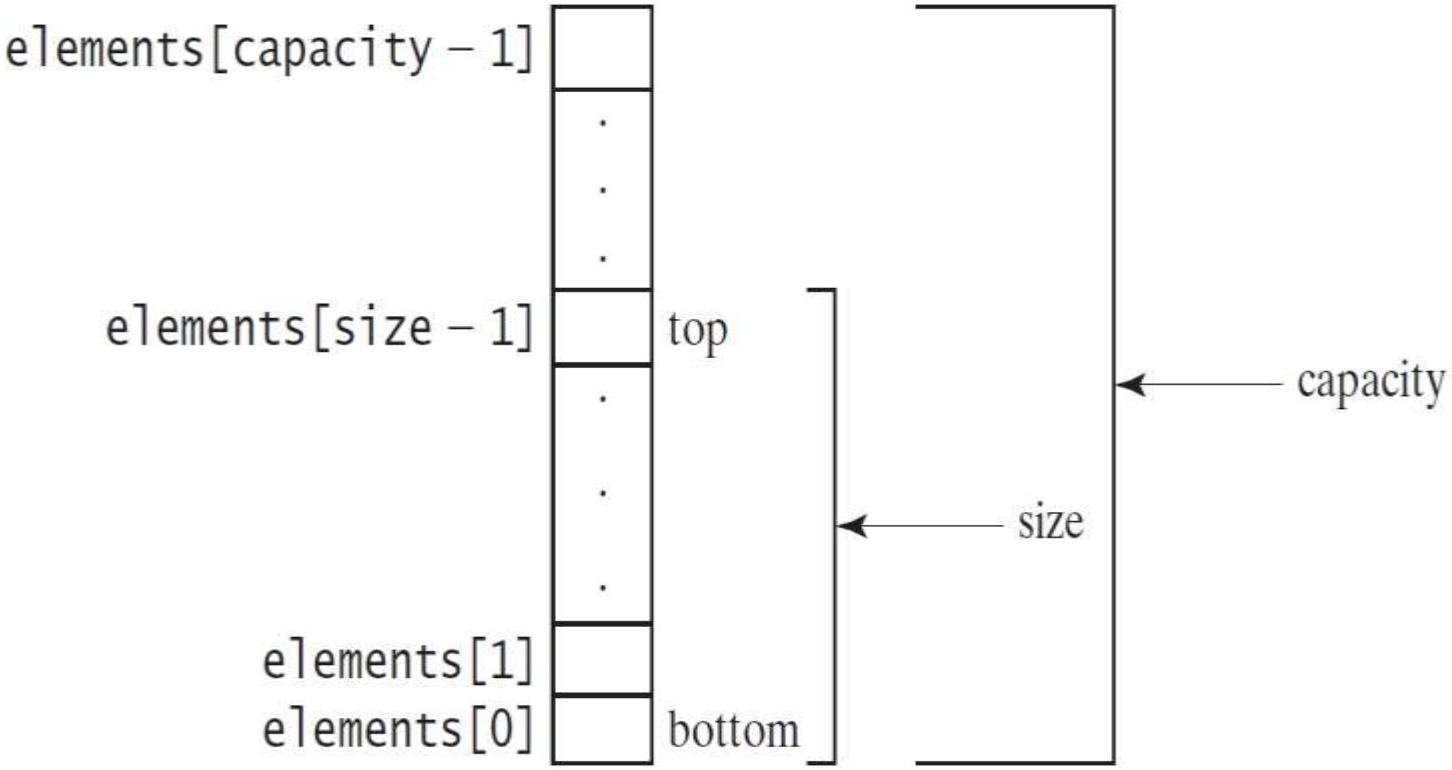


Designing the StackOfIntegers Class

a *stack* is a data structure that holds data in a last-in, first-out fashion

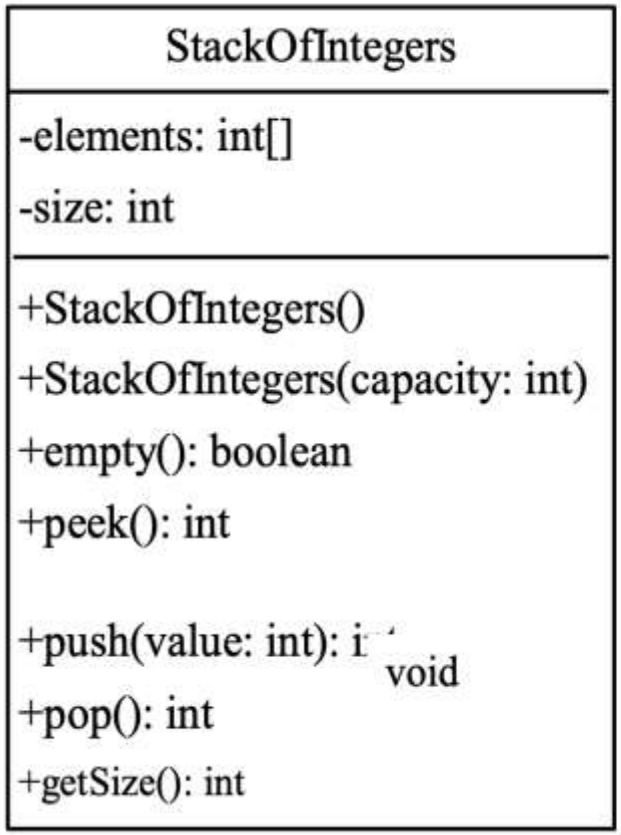


Implementing `StackOfIntegers` Class



StackOfIntegers

Example: The `StackOfIntegers` Class



An array to store integers in the stack.

The number of integers in the stack.

Constructs an empty stack with a default capacity of 16.

Constructs an empty stack with a specified capacity.

Returns true if the stack is empty.

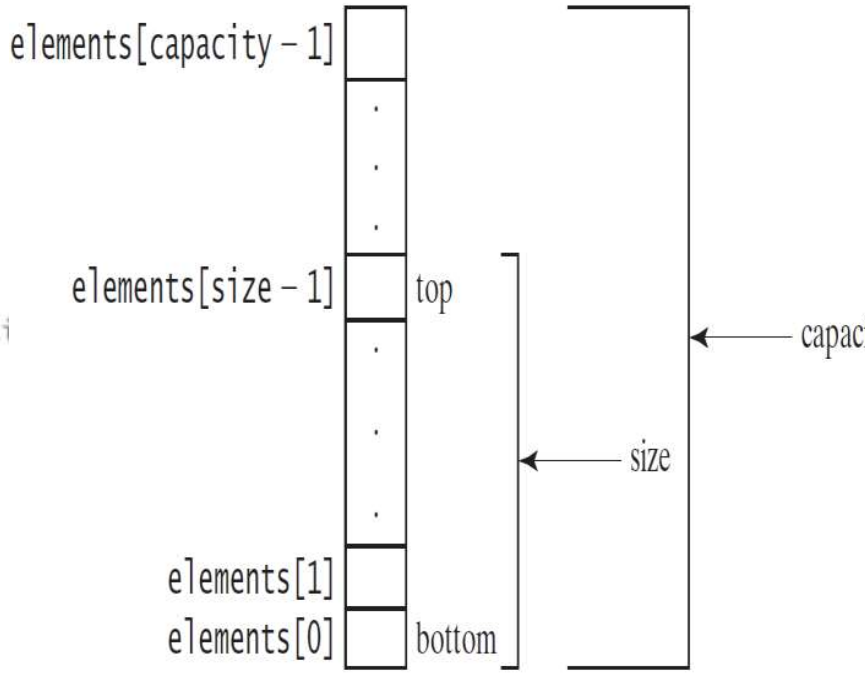
Returns the integer at the top of the stack without removing it from the stack.

Stores an integer into the top of the stack.

Removes the integer at the top of the stack and returns it.

Returns the number of elements in the stack.

```java
public class StackOfIntegers {
  private int[] elements;
  private int size;
  public static final int DEFAULT_CAPACITY = 16;

  /** Construct a stack with the default capacity 16 */
  public StackOfIntegers() {
    this (DEFAULT_CAPACITY);
  }

  /** Construct a stack with the specified maximum capaci
  public StackOfIntegers(int capacity) {
    elements = new int[capacity];
  }

  /** Push a new integer to the top of the stack */
  public void push(int value) {
    if (size >= elements.length) {
      int[] temp = new int[elements.length * 2];
      System.arraycopy(elements, 0, temp, 0, elements.length);
      elements = temp;
    }

    elements[size++] = value;
  }

  /** Return and remove the top element from the stack */
  public int pop() {
    return elements[--size];
  }

  /** Return the top element from the stack */
  public int peek() {
    return elements[size - 1];
  }

  /** Test whether the stack is empty */
  public boolean empty() {
    return size == 0;
  }

  /** Return the number of elements in the stack */
  public int getSize() {
    return size;
  }
}
```




```java
public class TestStackOfIntegers {
  public static void main(String[] args) {
    StackOfIntegers stack = new StackOfIntegers();

    for (int i = 0; i < 10; i++)
      stack.push(i); // Push i to the stack

    while (!stack.empty()) // Test if stack is empty
      System.out.print(stack.pop() + " ");
        // Remove and return from stack
  }
}
```

What is the output?

```
public class TestStackOfIntegers {
public static void main(String[] args) {
StackOfIntegers stack = new StackOfIntegers();
for (int i = 0; i < 10; i++)
    stack.push(i);
while (!stack.empty()) {
    System.out.print(stack.pop() + " ")
}
```

9 8 7 6 5 4 3 2 1 0

- What we do if we want to calculate an equation depending on the **maximum** number on an **intger** range????

```
int sum = 0;
for (int i = 0; i < 1000000; i++) {
   sum += 100000;   // overflow
}
```

int	-2,147,483,648	2,147,483,647

- What we do if the **intger** number will be entered as a string???

```
String s = ”123";      s= s+ 8;
System.out.println(s);   //   s = 1238 (string)
```

Wrapper Classes

```
String s = ”123";
int x = Integer.parseInt(s);  x =x+8;
System.out.println(x);      // s = 131  (num)
```

Wrapper Classes

```
Integer x = 5;
System.out.println(x);
// 5
```

Processing Primitive Data Type Values as Objects

- *A primitive type value (int ,long ,double, char,…) is not an object, but it can be wrapped in an object using wrapper classes in the Java API.*

	int (Primitive)	Integer (Wrapper)
Declare	int i = 5;	Integer x = 5; (Autoboxing) ……..
store	Direct value	object in Heap
Accept null?	int i = null; ✗	Integer x = null; ✓
Comparison	int a=5, b=5; a==b; // true	Integer a=200 , b=200; a==b; //(ref) false a.equals(b); // true
Use methods	int i=10; i.compareTo(5); ✗	Integer x=10; x.compareTo(5); ✓
Max and Min	int.MAX_VALUE ✗	Integer.MAX_VALUE ✓
Convert to String	✗	Integer.parseInt("123"); ✓
Autoboxing	✗	Integer x = 5; ✓
Unboxing	✗	Integer x = 5; int y = x; ✓

Wrapper Classes

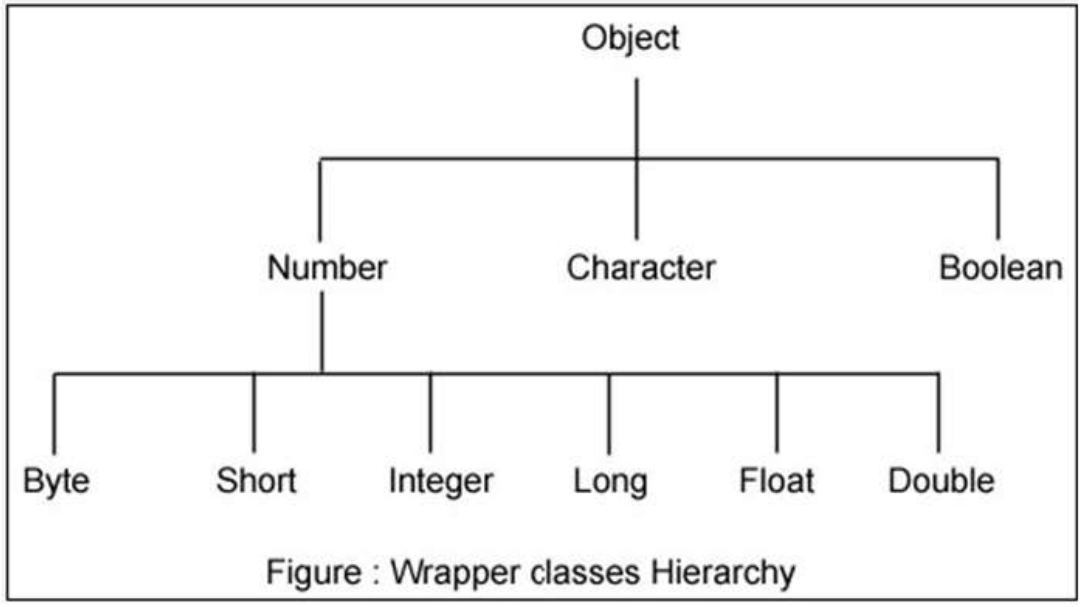


Figure : Wrapper classes Hierarchy

Let's work on the common Class

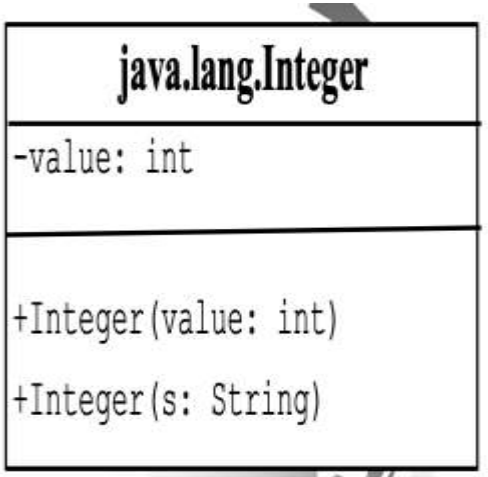
Integer

The `Integer` Class Constructors

- Integer x= 5; System.out.println(x); // 5
- Integer x= new Integer (5); // (Integer x= Integer.valueOf (5);)
  System.out.println(x); // 5
- Integer x= new Integer ("5"); // Integer x= Integer.valueOf("5"); (String)
  System.out.println(x); // 5

Does the Wrapper classes have no-arg constructors?? Integer x= new Integer ( ); ✕ **NO**

The creation is similar to the String Class, so what is common between them ??



The instances of all wrapper classes are **immutable**, i.e., their internal values cannot be changed once the objects are created.

The `Integer` and `Double` Classes

Numeric Wrapper Class Conversion Methods

java.lang.Integer
-value: int +MAX_VALUE: int +MIN_VALUE: int
+Integer(value: int) +Integer(s: String) +byteValue(): byte +shortValue(): short +intValue(): int +longVlaue(): long +floatValue(): float +doubleValue():double +compareTo(o: Integer): int +toString(): String +valueOf(s: String): Integer +valueOf(s: String, radix: int): Integer +parseInt(s: String): int +parseInt(s: String, radix: int): int

java.lang.Double
-value: double +MAX_VALUE: double +MIN_VALUE: double
+Double(value: double) +Double(s: String) +byteValue(): byte +shortValue(): short +intValue(): int +longVlaue(): long +floatValue(): float +doubleValue():double +compareTo(o: Double): int +toString(): String +valueOf(s: String): Double +valueOf(s: String, radix: int): Double +parseDouble(s: String): double +parseDouble(s: String, radix: int): double

Constructors and Constants

- You can construct a wrapper object either from a primitive data type value or from a string representing the numeric value—for example, **new Double(5.0)**, **new Double("5.0")**, **new Integer(5)**, and **new Integer("5")**.
- The wrapper classes do not have no-arg constructors. The instances of all wrapper classes are immutable; this means that, once the objects are created, their internal values cannot be changed.
- Each numeric wrapper class has the constants **MAX_VALUE** and **MIN_VALUE**.
- **MAX_VALUE** represents the maximum value of the corresponding primitive data type.
- System.out.println(Integer.MAX_VALUE); // 2147483647
- The minimum positive float (1.4E-45),
- The maximum double floating-point number (1.79769313486231570e+308d).

Conversion methods

- Each numeric wrapper class contains the methods **doubleValue()**, **floatValue()**, **intValue()**, **longValue()**, and **shortValue()** for returning Primitive **double**, **float**, **int**, **long**, or **short** value for the wrapper object.
- For example,

Integer num = 12;

num.doubleValue() returns **12.0**

new Double(12.4).intValue() returns **12**

compareTo method

- Recall that the **String** class contains the **compareTo** method for comparing two strings.
- The numeric wrapper classes contain the **compareTo** method for comparing two numbers and returns **1**, **0**, or **-1**, if this number is greater than, equal to, or less than the other number.
- **Integer num = 10;**

  System.out.println(num.compareTo(9)); returns **1**

- **new Double(12.3).compareTo(new Double(12.3))** returns **0**
- **new Double(12.3).compareTo(new Double(12.51))** returns **-1**

static valueOf methods

- The numeric wrapper classes have a useful static method, **valueOf (String s)**.
- This method creates a new object initialized to the value represented by the specified string.
- For example,

```
Double doubleObject = Double.valueOf("12.4"); // 12.4
Integer obj;
        obj= Integer.valueOf("12"); // 12
```

static parsing methods

- The **parseInt** method in the **Integer** class to parse a numeric string into an **int** value (primitive type) and the **parseDouble** method in the **Double** class to parse a numeric string into a **double** value.
- Each numeric wrapper class has two overloaded parsing methods to parse a numeric string into an appropriate numeric value based on **10** (decimal) or any specified radix (e.g., **2** for binary, **8** for octal, and **16** for hexadecimal).

- Integer.parseInt("11") returns 11
- Integer.parseInt("11", 2) returns 3;
- Integer.parseInt("10", 2) returns 2;
- Integer.parseInt("12", 2) would raise a runtime exception because 12 is not a binary numbe
- Integer.parseInt("12", 8) returns 10;
- Integer.parseInt("13", 10) returns 13;

- Integer.parseInt("1A", 16) returns 26;

Note that you can convert a decimal number into a hex number using the format method. For example,

String.format("%x", 26) returns 1a;

Automatic Conversion between Primitive Types and Wrapper Class Types

- *A primitive type value can be automatically converted to an object using a wrapper class, and vice versa, depending on the context.*
- Converting a primitive value to a wrapper object is called *boxing*.
- The reverse conversion is called *unboxing*.
- The compiler will automatically box a primitive value that appears in a context requiring an object and will unbox an object that appears in a context requiring a primitive value.

Automatic Conversion Between Primitive Types and Wrapper Class Types

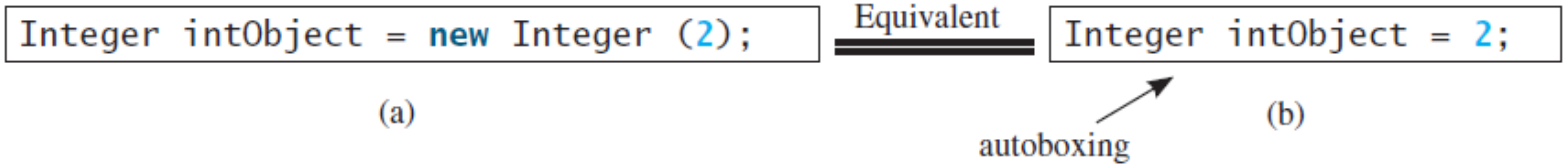
- *Autoboxing* is the automatic conversion of a primitive value to a corresponding wrapper object:

```
Integer obj;
int num = 42;
obj = num;
```

- The assignment creates the appropriate `Integer` object
- The reverse conversion (called *unboxing*) also occurs automatically as needed

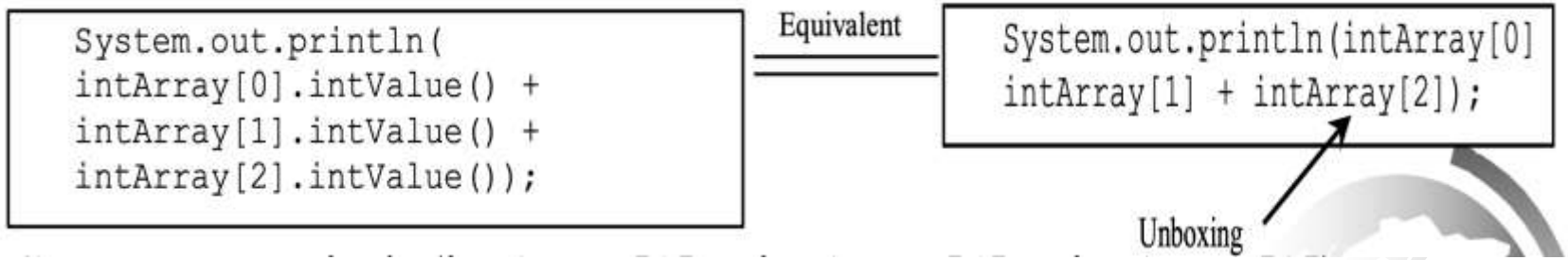
```
Integer obj=42;
int num;
num = obj;
```

Automatic Conversion between Primitive Types and Wrapper Class Types



- Integer[] intArray = {**1**, **2**, **3**};

the primitive values **1**,**2**, and **3** are automatically boxed into objects **new Integer(1)**, **new Integer(2)**, and **new Integer(3)**.



System.out.println(intArray[**0**] + intArray[**1**] + intArray[**2**]);
the objects **intArray[0]**, **intArray[1]**, and **intArray[2]** are automatically unboxed into **int** values that are added together.

The **BigInteger** and **BigDecimal** Classes

- If you need to compute with very large integers or high-precision floating-point values, you can use the **BigInteger** and **BigDecimal** classes.
- An instance of **BigInteger** can represent an integer of any size.
- **In the java.math package**
- Both are *immutable*. Both extend the Number class and implement the Comparable interface.

BigInteger and BigDecimal

```
BigInteger a = new BigInteger("92233720368547758O7"); // Should be String
BigInteger b = new BigInteger("2");
BigInteger c = a.multiply(b); // 92233720368547758O7 * 2
System.out.println(c);
```

Output = 18446744073709551614

```
a.add(b);   a.subtract(b);   a.multiply(b);   a.divide(b);
```

```
BigDecimal a = new BigDecimal(1.0); //1.0
BigDecimal b = new BigDecimal(3); //3

BigDecimal c = a.divide(b, 20, BigDecimal.ROUND_UP);
System.out.println(c);
```

Rounding mode

Scale is the max no. of digits after the decimal point

Output = 0.33333333333333333334

Is there a way to convert a number to binary???

- Wrapper classes contain methods that help manage the associated type

```
static String toBinaryString ( int num);
static String tohexString ( int num);
static String toOctalString ( int num);
```

Return a string representation

EX : System.out.print (Integer.toBinaryString(6)); //110